

Laboratorio 2.1: Esquemas de Detección y Corrección de Errores

Integrantes:

Mathew Cordero – 22982

Javier Chen – 22153

Descripción de la práctica:

En este laboratorio implementamos dos algoritmos para detección y corrección de errores en la capa de enlace.

Para detección de errores usamos CRC-32

Para corrección de errores código de Hamming.

Pruebas:

Receptor HammingReceptor.java y emisorhamming.py como emisor

Sin Errores

.java

```
PS E:\Laboratorio2-Redes> e.; cd 'e:\Laboratorio2-Redes'; & 'C:\Program Files\Java\jdk-22\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\285bcd589a94a9673f0b9a23f5d2d6e\redhat.java\jdt_ws\Laboratorio2-Redes\27c198b2\bin' 'HammingReceptor'
Ingrese la trama recibida de 7 bits (ej. 1101001): 0110011
Trama recibida: 0110011
Síndrome de error: 0
? No se detectaron errores

--- Resultado del Receptor Hamming(7,4) ---
Mensaje decodificado (4 bits): 1011
```

.py



Ingrese un mensaje de 4 bits (ej. 1011): 1011

```
--- Resultado del Emisor Hamming(7,4) ---  
Mensaje original: 1011  
Trama codificada (7 bits): 0110011
```

```
sages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\4  
285bcd589a94a9673f0b9a23f5d2d6e\redhat.java\jdt_ws\Laboratorio2-Redes_  
27c198b2\bin' 'HammingReceptor'  
3f0b9a23f5d2d6e\x5credhat.java\x5cjdt_ws\x5cLaboratorio2-Redes_27c198b  
2\x5cbin' 'HammingReceptor' ;465764cc-e75d-4754-ab81-d765f5e3325aIngre  
se la trama recibida de 7 bits (ej. 1101001): 1100110  
Trama recibida: 1100110  
Síndrome de error: 0  
? No se detectaron errores  
● --- Resultado del Receptor Hamming(7,4) ---  
Mensaje decodificado (4 bits): 0110
```

.py



Ingrese un mensaje de 4 bits (ej. 1011): 0110

```
--- Resultado del Emisor Hamming(7,4) ---  
Mensaje original: 0110  
Trama codificada (7 bits): 1100110
```

.java

```
sages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\4  
285bcd589a94a9673f0b9a23f5d2d6e\redhat.java\jdt_ws\Laboratorio2-Redes_  
27c198b2\bin' 'HammingReceptor'  
3f0b9a23f5d2d6e\x5credhat.java\x5cjdt_ws\x5cLaboratorio2-Redes_27c198b  
2\x5cbin' 'HammingReceptor' ;465764cc-e75d-4754-ab81-d765f5e3325aIngre  
se la trama recibida de 7 bits (ej. 1101001): 0111100  
Trama recibida: 0111100  
Síndrome de error: 0  
● ? No se detectaron errores  
● --- Resultado del Receptor Hamming(7,4) ---  
Mensaje decodificado (4 bits): 1100
```

.py

Ingrese un mensaje de 4 bits (ej. 1011): 1100

--- Resultado del Emisor Hamming(7,4) ---

Mensaje original: 1100

Trama codificada (7 bits): 0111100

Con un Error

Mensaje original sin codificar: 1011

Mensaje original codificado: 0110011

Trama con error en bit 3: 0100011

```
27c198b2\bin' 'HammingReceptor'
3f0b9a23f5d2d6e\x5credhat.java\x5cjd_t_ws\x5cLaboratorio2-Redes_27c198
2\x5cbn' 'HammingReceptor' ;465764cc-e75d-4754-ab81-d765f5e3325aIngr
se la trama recibida de 7 bits (ej. 1101001): 0100011
Trama recibida: 0100011
Síndrome de error: 3
? Error detectado en la posición: 3
? Error corregido
Trama corregida: 0110011

--- Resultado del Receptor Hamming(7,4) ---
Mensaje decodificado (4 bits): 1011
```

Mensaje original sin codificar: 0110

Mensaje original codificado: 1100110

Trama con error en bit 2: 1000110

```
27c198b2\bin' 'HammingReceptor'
3f0b9a23f5d2d6e\x5credhat.java\x5cjd_t_ws\x5cLaboratorio2-Redes_27c198b2\x5cbn' 'HammingReceptor' ;465764cc-
Ingrese la trama recibida de 7 bits (ej. 1101001): 1000110
Trama recibida: 1000110
Síndrome de error: 2
? Error detectado en la posición: 2
? Error corregido
Trama corregida: 1100110

--- Resultado del Receptor Hamming(7,4) ---
Mensaje decodificado (4 bits): 0110
```

Mensaje original sin codificar: 1100

Mensaje original codificado: 0111100

Trama con error en bit 7: 0111101

```
27c198b2\bin' 'HammingReceptor'
3f0b9a23f5d2d6e\x5credhat.java\x5cjdk_ws\x5cLaboratorio2-Redes_27c198b
2\x5cb' 'HammingReceptor' ;465764cc-e75d-4754-ab81-d765f5e3325aIngre
se la trama recibida de 7 bits (ej. 1101001): 0111101
Trama recibida: 0111101
Síndrome de error: 7
? Error detectado en la posición: 7
? Error corregido
Trama corregida: 0111100

--- Resultado del Receptor Hamming(7,4) ---
Mensaje decodificado (4 bits): 1100
```

Receptor CRC- 32 Python y CRC32Emisor.java como emisor

Sin Errores

.java

```
am Files\Java\jdk-22\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMes
sages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\4
285bcd589a94a9673f0b9a23f5d2d6e\redhat.java\jdk_ws\Laboratorio2-Redes_
27c198b2\bin' 'CRC32Emisor'
3f0b9a23f5d2d6e\x5credhat.java\x5cjdk_ws\x5cLaboratorio2-Redes_27c198b
2\x5cb' 'CRC32Emisor' ;1996b233-79b8-4d5c-9602-c0411684a8f9Emisor CR
C-32 (Generación de trama con CRC)
Ingrese el mensaje binario (sin CRC): 1011
Trama generada (mensaje + CRC):
101100101011010010111100101101100001
```

.py

```
Receptor CRC-32 (Detección de errores)
Ingrese la trama recibida (mensaje + CRC de 32 bits): 101101010110100101111001011011000010
Trama válida. No se detectaron errores.
Mensaje recibido: 1011
```

.java

```
PS E:\Laboratorio2-Redes> e.; cd 'e:\Laboratorio2-Redes'; & 'C:\Program Files\Java\jdk-22\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\4285bcd589a94a9673f0b9a23f5d2d6e\redhat.java\jdt_ws\Laboratorio2-Redes_27c198b2\bin' 'CRC32Emisor' 3f0b9a23f5d2d6e\x5credhat.java\x5cjdt_ws\x5cLaboratorio2-Redes_27c198b2\x5cbins' 'CRC32Emisor' ;1996b233-79b8-4d5c-9602-c0411684a8f9Emisor CRC-32 (Generación de trama con CRC)
Ingrese el mensaje binario (sin CRC): 1101011011
Trama generada (mensaje + CRC):
11010110110010000100110111001111110110101
PS E:\Laboratorio2-Redes>
```

.py

```
Receptor CRC-32 (Detección de errores)
Ingrese la trama recibida (mensaje + CRC de 32 bits): 11010110110010000100110111001111110110101
Trama válida. No se detectaron errores.
Mensaje recibido: 1101011011
```

.java

```
PS E:\Laboratorio2-Redes> e.; cd 'e:\Laboratorio2-Redes'; & 'C:\Program Files\Java\jdk-22\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\4285bcd589a94a9673f0b9a23f5d2d6e\redhat.java\jdt_ws\Laboratorio2-Redes_27c198b2\bin' 'CRC32Emisor' 3f0b9a23f5d2d6e\x5credhat.java\x5cjdt_ws\x5cLaboratorio2-Redes_27c198b2\x5cbins' 'CRC32Emisor' ;1996b233-79b8-4d5c-9602-c0411684a8f9Emisor CRC-32 (Generación de trama con CRC)
Ingrese el mensaje binario (sin CRC): 1010101111001101
Trama generada (mensaje + CRC):
101010111100110100110110110010111110010001110101
PS E:\Laboratorio2-Redes>
```

.py

```
Receptor CRC-32 (Detección de errores)
Ingrese la trama recibida (mensaje + CRC de 32 bits): 101010111100110100110110110010111110010001110101
Trama válida. No se detectaron errores.
Mensaje recibido: 1010101111001101
```

Con un Error

Mensaje original: 1011

Mensaje original: 101100101011010010111100101101100001

Trama con error en bit 5: 101101101011010010111100101101100001



Receptor CRC-32 (Detección de errores)

Ingrese la trama recibida (mensaje + CRC de 32 bits): 101101101011010010111100101101100001

Error detectado en la trama. Número aproximado de bits erróneos: 1. Se descarta el mensaje.

Mensaje original: 1101011011

Trama sin error: 11010110110010000100110111001111110110101

Trama con error en bit 6: 11010010110010000100110111001111110110101

Receptor CRC-32 (Detección de errores)

Ingrese la trama recibida (mensaje + CRC de 32 bits): 11010010110010000100110111001111110110101

Error detectado en la trama. Se descarta el mensaje.

Mensaje original: 1010101111001101

Trama sin error: 101010111100110100110110110010111110010001110101

Trama con error en el bit 2: 111010111100110100110110110010111110010001110101

Receptor CRC-32 (Detección de errores)

Ingrese la trama recibida (mensaje + CRC de 32 bits): 111010111100110100110110110010111110010001110101

Error detectado en la trama. Se descarta el mensaje.

Dos errores o mas

Mensaje original: 1011

Trama sin error: 101100101011010010111100101101100001

Trama con errores en bits [1, 4]: 111110101011010010111100101101100001

Receptor CRC-32 (Detección de errores)

Ingrese la trama recibida (mensaje + CRC de 32 bits): 111110101011010010111100101101100001

Error detectado en la trama. Se descarta el mensaje.

Mensaje original: 1101011011

Trama sin error: 11010110110010000100110111001111110110101

Trama con errores en bits [2, 7, 10]:

11110111111010000100110111001111110110101

Receptor CRC-32 (Detección de errores)

Ingrese la trama recibida (mensaje + CRC de 32 bits): 11110111111010000100110111001111110110101

Error detectado en la trama. Se descarta el mensaje.

Mensaje original: 1010101111001101

Trama sin error: 101010111100110100110110110010111110010001110101

Trama con errores en bits [0, 5, 15, 20]:

0010111111100110000111110110010111110010001110101

Receptor CRC-32 (Detección de errores)

Ingrese la trama recibida (mensaje + CRC de 32 bits): 0010111111100110000111110110010111110010001110101

Error detectado en la trama. Se descarta el mensaje.

Preguntas (Hamming):

¿Es posible manipular los bits de tal forma que el algoritmo seleccionado no sea capaz de detectar el error? ¿Por qué sí o por qué no? En caso afirmativo, demuéstrelo con su implementación.

Sí, es posible manipular los bits de tal forma que el algoritmo Hamming no detecte el error, pero solo en casos de errores múltiples. El código Hamming solo detecta y corrige errores de un solo bit.

Si se cambian dos o más bits en la misma trama, es posible que:

- Valor que indica la posición del error no corresponda a ninguna posición real del error.
- El algoritmo corrija un bit que no está dañado, haciendo que el mensaje resultante sea incorrecto sin saberlo.

En base a las pruebas que realizó, ¿qué ventajas y desventajas posee cada algoritmo con respecto a los otros dos? Tome en cuenta complejidad, velocidad, redundancia (overhead), etc.

Ventajas de Hamming:

- Capacidad de corrección: puede corregir errores de 1 solo bit sin necesidad de retransmisión.
- Velocidad: es muy rápido para mensajes pequeños, ya que solo requiere operaciones de paridad simples.
- Implementación sencilla tanto en software como en hardware.
- Ideal para entornos donde los errores de 1 bit son frecuentes (como en memoria RAM o sistemas embebidos).

Desventajas:

- No detecta ni corrige errores múltiples (2 o más bits), lo que puede causar errores no detectados.

- La redundancia crece rápidamente al aumentar el tamaño del mensaje, lo que lo hace poco eficiente para mensajes largos.
- Solo detecta errores de 2 bits si el código se ha extendido, pero no los puede corregir.

Preguntas (CRC- 32):

¿Es posible manipular los bits de tal forma que el algoritmo seleccionado no sea capaz de detectar el error? ¿Por qué sí o por qué no? En caso afirmativo, demuéstrelo con su implementación.

Sí, es posible, aunque muy poco probable, que se manipulen los bits de un mensaje y que el algoritmo no detecte el error. Esto se debe a que CRC-32, aunque muy robusto como código de detección, no es infalible: puede haber colisiones, es decir, errores que modifican la trama pero que resultan en el mismo residuo CRC.

En base a las pruebas que realizó, ¿qué ventajas y desventajas posee cada algoritmo con respecto a los otros dos? Tome en cuenta complejidad, velocidad, redundancia (overhead), etc.

Ventajas de CRC-32:

- Alta capacidad de detección de errores, especialmente:
 - Todos los errores de 1 bit y 2 bits.
 - Todos los errores en ráfagas de hasta 32 bits.
- Bajo overhead en tramas largas (solo 32 bits de redundancia).
- Rápido y eficiente, especialmente en hardware o con operaciones a nivel de bits.
- Muy usado en redes (Ethernet, USB) y almacenamiento digital (discos, ZIP, etc.).

Desventajas respecto a otros algoritmos (como Hamming):

- No corrige errores, solo los detecta.
- No es útil para mensajes muy cortos, ya que el overhead fijo (32 bits) es alto proporcionalmente.
- Puede haber colisiones: errores múltiples muy específicos que no son detectados (aunque son raros).